

Exploratory Data Analysis (EDA): Bivariate and Multivariate Analysis

1. Introduction

In previous sessions, we focused on **Univariate Analysis** (analyzing one column at a time). Today, we move to analyzing multiple columns together to find relationships between them.

Key Definitions:

- **Bivariate Analysis:** Analyzing the relationship between **two** variables/columns.
- **Multivariate Analysis:** Analyzing the relationship between **more than two** variables/columns (e.g., how Age and Gender together affect Survival).

2. Setup and Datasets

The teacher uses four different datasets because different techniques work better on different types of data.

Libraries Used:

```
import pandas as pd  
  
import seaborn as sns
```

Datasets Used:

1. **Tips Dataset:** Restaurant data (Total bill, tip, gender, etc.).
2. **Titanic Dataset:** Information about passengers (Survived, Age, Pclass, etc.).
3. **Flights Dataset:** Monthly passenger count for US airlines (Year, Month, Passengers).
4. **Iris Dataset:** Measurements of different flower species (Petal length, Sepal length, etc.).

3. Numerical vs. Numerical Analysis

When both columns contain numbers (e.g., Total Bill and Tip), we use a **Scatter Plot**.

Concept: Scatter Plot

A scatter plot shows how one numerical variable changes in relation to another. Each dot represents one observation (e.g., one customer).

Code: Basic Bivariate Analysis

```
sns.scatterplot(tips['total_bill'], tips['tip'])
```

Line-by-line explanation:

- `sns.scatterplot()`: Tells Seaborn to create a scatter plot.
- `tips['total_bill']`: Sets the X-axis to the total bill amount.
- `tips['tip']`: Sets the Y-axis to the tip amount.

Observation: There is a **Linear Relationship**. As the total bill increases, the tip generally increases.

Multivariate Analysis using Scatter Plot

We can add more dimensions to a 2D scatter plot using **Hue**, **Style**, and **Size**.

A. Using 'Hue' (Adding a 3rd Variable)

```
sns.scatterplot(tips['total_bill'], tips['tip'], hue=tips['sex'])
```

Line-by-line explanation:

- `hue=tips['sex']`: Colors the dots based on gender (e.g., Blue for Male, Orange for Female).
- **Insight**: We can now see if males or females tip differently for the same bill amount.

B. Using 'Style' (Adding a 4th Variable)

```
sns.scatterplot(tips['total_bill'], tips['tip'], hue=tips['sex'], style=tips['smoker'])
```

Line-by-line explanation:

- `style=tips['smoker']`: Changes the shape of the marker (e.g., 'X' for smokers, 'O' for non-smokers).
- **Insight**: Now we are tracking Bill, Tip, Gender, and Smoking status in one graph.

C. Using 'Size' (Adding a 5th Variable)

```
sns.scatterplot(tips['total_bill'], tips['tip'], hue=tips['sex'], style=tips['smoker'], size=tips['size'])
```

Line-by-line explanation:

- `size=tips['size']`: Makes the dots larger or smaller based on the number of people at the table.
- **Insight**: Larger dots mean more people were sitting together.

4. Numerical vs. Categorical Analysis

Used when one column is a number (Age) and the other is a category (Gender/Class).

A. Bar Plot (Average Value)

By default, a bar plot shows the **mean (average)** value for each category.

```
sns.barplot(titanic['Pclass'], titanic['age'])
```

Line-by-line explanation:

- `titanic['Pclass']`: X-axis (Categorical - 1st, 2nd, 3rd class).
- `titanic['age']`: Y-axis (Numerical).
- **Insight**: This shows the average age of people in each class. Usually, Class 1 has older people.

Adding Multivariate info:

```
sns.barplot(titanic['Pclass'], titanic['age'], hue=titanic['sex'])
```

- **Insight:** Shows the average age of males vs. females within each travel class.

B. Box Plot (Distribution)

Box plots are great for seeing the "spread" of data and identifying **outliers**.

```
sns.boxplot(titanic['sex'], titanic['age'], hue=titanic['survived'])
```

Line-by-line explanation:

- `titanic['sex']`: Categories on X-axis.
- `titanic['age']`: Numerical values on Y-axis.
- `hue=titanic['survived']`: Splits boxes based on who lived or died.
- **Insight:** We can see the age distribution of survivors vs. non-survivors for both genders.

C. Distplot / KDE Plot (Probability Analysis)

This is used to see the "density" or probability of an event happening across a range of numbers.

Teacher's Example: Survival by Age

```
sns.distplot(titanic[titanic['survived']==0]['age'], hist=False)
```

```
sns.distplot(titanic[titanic['survived']==1]['age'], hist=False)
```

Explanation:

- We filter the data into two groups: those who died (`survived==0`) and those who lived (`survived==1`).
- `hist=False`: We only want the curve (PDF), not the bars.
- **Storytelling Insight:**
 - If the curves overlap, age didn't matter.
 - **Finding:** At very young ages (children), the "Survived" curve is higher than the "Died" curve. This means children had a higher probability of being saved.

5. Categorical vs. Categorical Analysis

Used when both columns are categories (e.g., `Pclass` and `Survived`).

A. Heatmap (using `Crosstab`)

First, we create a table (Matrix) showing counts, then visualize it.

```
pd.crosstab(titanic['Pclass'], titanic['survived'])
```

Explanation: This creates a table showing how many people from each class survived or died.

Visualization with Heatmap:

```
sns.heatmap(pd.crosstab(titanic['Pclass'], titanic['survived']))
```

- **Insight:** Colors show where the density is highest. Darker/Lighter colors (depending on palette) indicate more deaths in Class 3.

B. Clustermap

A clustermap is a heatmap that also groups similar rows/columns together using **Dendrograms** (tree-like structures).

```
sns.clustermap(pd.crosstab(titanic['sibsp'], titanic['survived']))
```

- **Why use it?** It tells you which categories behave similarly. For example, people traveling with 5 or 8 siblings might have similar survival rates.

6. Pair Plot

If your dataset has many numerical columns (like Iris), instead of making 10 separate scatter plots, you use a **Pair Plot**.

```
sns.pairplot(iris, hue='species')
```

Line-by-line explanation:

- `sns.pairplot(iris)`: Automatically creates a grid of scatter plots for every numerical column against every other numerical column.
- **Diagonal**: When a column meets itself (e.g., Petal Width vs. Petal Width), it plots a Histogram.
- `hue='species'`: Colors the dots by flower type.
- **Insight**: In one glance, you can see which measurements best separate the flower species.

7. Line Plot (Time Series)

Line plots are used specifically when the X-axis represents **Time** (Years, Months, Days).

```
new_df = flights.groupby('year').sum().reset_index()
```

```
sns.lineplot(new_df['year'], new_df['passengers'])
```

Line-by-line explanation:

- `groupby('year').sum()`: Totals the passengers for each year.
- `sns.lineplot()`: Connects the points with a line.
- **Insight**: This shows the growth trend of the airline industry over time.

Summary Cheat Sheet for Students

Analysis Type	Plot to Use	Use Case
Num vs Num	Scatter Plot	To see correlation/trends between two numbers.
Num vs Cat	Bar / Box / Distplot	To compare averages or distributions across categories.
Cat vs Cat	Heatmap / Crosstab	To see frequencies/percentages between two categories.

Many Num Columns	Pair Plot	To see all possible relationships at once.
-------------------------	-----------	--

Num vs Time	Line Plot	To see trends over time.
--------------------	-----------	--------------------------

Important Tips from the Teacher:

1. **Don't just look at numbers:** Convert counts to percentages (e.g., use `mean()` on 'survived' column) to get a true picture of probability.
2. **Storytelling is Key:** The goal of EDA is not just to make graphs, but to find the "Story" (e.g., "Why did more people survive in Class 1?").
3. **Automation:** While these tools are manual, the next step is learning how to automate these insights.